

Transpilation Meets the B Method

Akram Idani^[0000–0003–2267–3639]

Univ. Grenoble Alpes, Grenoble INP, CNRS, F-38000 Grenoble France
`akram.idani@univ-grenoble-alpes.fr`

Abstract. Transpilers are popular tools for bridging languages or notations through grammar-to-grammar translations. They can be seen as lightweight compilers; however, they are rarely developed within a formal framework, which makes their semantic correctness difficult to guarantee. By contrast, compilers benefit from more rigorous verification techniques, at the cost of significantly heavier infrastructures. In this work, we introduce a proof-based framework for transpilation built on the B Method. In our approach, transpilation rules are specified as B machines, which generate proof obligations ensuring that the translation preserves the intended semantics. This framework is implemented in **Meeduse**, a language workbench we developed to provide formal semantics, together with execution and debugging facilities for domain-specific languages. As a case study, we revisit the UML-to-B translation and use **Meeduse** as the execution environment for the underlying mappings.

Keywords: Transpilation · B Method · Theorem Proving · DSLs

1 Introduction

Preamble. I had the privilege of meeting Jean-Raymond Abrial in 2004 while I was a PhD student. At that time, UML-B was attracting growing attention within the B community. Combining formal methods with structuring notations can help bridge the gap between rigorous semantics and domain modeling. Jean-Raymond agreed with this research direction and emphasized that B was not only a method for system design, but also a means to reformulate domain requirements in terms of formal properties. Since then, my research has often been in line with this idea, particularly through the development of tools that transpile domain-specific notations into B models. This chapter further develops this vision by showing how B can serve not only as a target formalism, but also as the formal framework in which transpilation is specified, proved, and executed.

Existing Language Workbenches (LWBs), do not support formal methods at all. While this may sound like a strong assertion, it is in fact grounded in the studies of Kosar et al. [13] and Iung et al. [12], who conducted a thorough analysis of LWBs from 2006 to 2019. The first study (2006-2012) highlighted an urgent need to identify the reasons for this lack and to propose suitable solutions. The second study (2012-2019) refers only to testing as a verification activity and shows that, among 59 tools analyzed, only 9 provided any support for testing.

In our work, we provide a pragmatic response to this issue by relying on the B Method [1] and its ecosystem of tools for verification, *i.e.*, theorem proving, animation and model checking. We have developed **Meeduse** [6], a LWB that enables the formal verification of a DSL by describing its semantics in B. A first proof of concept was presented in 2019 [10] as an application case in the domain of Railway Systems; the tool was still at an embryonic stage. Since then, it has evolved, gained maturity, and been the subject of several publications [7,8,9]. It has also been successfully applied to several case studies and won two awards at the Transformation Tool Contest [11]: best verification and audience award.

The choice of the B method as the semantic space of **Meeduse** is motivated by the strong structural and semantic compatibility between meta-modeling formalisms and B. In **Meeduse** the semantics of DSLs is defined by means of meta-models, as standardized by the OMG through the Meta Object Facility (MOF) and constrained with OCL, which is fundamentally built on sets, relations, and first-order logic. These are precisely the mathematical foundations of B, which means it can describe systems at a comparable level of abstraction. The key difference is that B goes one step further by equipping the formal model with generalized substitutions, and therefore with an executable semantics.

In this chapter, we connect the **Meeduse** approach to language transpilation. We argue that a rigorous transpilation process requires the formal modeling of both the source and the target languages, as well as the mappings between them. **Meeduse** provides such a setting, where both languages can be specified with a B-based semantics, and the transpilation rules can be written in B and proven correct while remaining executable. To demonstrate our approach we apply it to revisit the transpilation of UML models into B specifications. Note that Two main paradigms coexist within the B family: Operational B [1] (often referred to simply as B) and Event-B [2]. Both paradigms are actively maintained: one primarily in an industrial context, supported by Atelier B and ClearSy [14], and the other predominantly within the research community, where the extensible Rodin framework [4] enables the integration of new modeling paradigms and theoretical extensions. As noted by J.-R. Abrial [3]: “*Event-B for the system analysis, B for the software development [...], most of the time, system engineers and software engineers have different cultures*”. This observation naturally raises the question of how DSL’s transpilation should be positioned: is it closer to system modeling or to software development? In this work, we adopt the B method as the semantic space for transpilation; our approach is therefore aligned with software development. However, we believe that the same principles could be applied to Event-B, which will be the focus of future work.

The remainder of this chapter is organized as follows. Section 2 provides an overview of the **Meeduse** LWB. Section 3 introduces the UML-to-B use case through a simple example, including both its UML model and the corresponding B specification. Section 4 then details the grammar-to-grammar transpilation process. Section 5 discusses related approaches, and Section 6 concludes the chapter and outlines perspectives for future work.

2 Quick overview of Meeduse

A key idea of *Meeduse* is to exploit the correspondence between a DSL grammar, its abstract syntax tree (AST), and a formal representation of this AST in B. Note that *Meeduse* builds on *XText*¹, which relies on *LL(*)* grammars and ANTLR for parsing, and the Eclipse Modeling Framework² for representing ASTs. Starting from a DSL grammar defined in *Xtext*, *Meeduse* generates a B machine, called the *static semantics*, which formalizes the abstract syntax of the language independently of any concrete model. Given a textual model conforming to the grammar, *Meeduse* then generates a refinement of the static semantics that represents this particular model. In this data refinement, the elements of the model, such as objects, identifiers, and attribute values, are introduced. Figure 1 illustrates this process on a simple language. On the left-hand side, a grammar defines a textual language for counters and their values. From this grammar, *Meeduse* extracts a B machine capturing the corresponding abstract syntax. On the right-hand side, a concrete model containing two counters is translated into a refinement in which these counters and their values are introduced.

DSL's Grammar <pre> Model : (Counter)* ; Counter : ID '←' value=INT ; </pre> MACHINE <i>MeeduseTuto</i> SETS <i>COUNTER</i> CONCRETE_CONSTANTS <i>Counter</i> PROPERTIES $Counter \in \mathcal{F}(COUNTER)$ CONCRETE_VARIABLES <i>value</i> INVARIANT $value \in Counter \rightarrow INT$ INITIALISATION $value : \in Counter \rightarrow INT$ END	Textual model <pre> Counter1 ← 5 Counter2 ← 3 </pre> REFINEMENT <i>MeeduseTuto_r</i> REFINES <i>MeeduseTuto</i> CONSTANTS COUNTER1, COUNTER2 PROPERTIES $COUNTER1 \in COUNTER$ $\wedge COUNTER2 \in COUNTER$ $\wedge \text{card}(\{COUNTER1, COUNTER2\}) = 2$ $\wedge Counter = \{COUNTER1, COUNTER2\}$ INITIALISATION $value := \{(COUNTER1 \mapsto 5),$ $\quad (COUNTER2 \mapsto 3)\}$ END
---	---

Fig. 1: A grammar and its instance in *Meeduse*

From this point on, the evolution of the model (*e.g.*, incrementing a counter) is defined by means of B operations, which represents the *dynamic semantics* of the DSL. These operations can be formally verified using B proof tools such as Atelier B. Regarding model execution, it relies on ProB[15]: any modification

¹ <https://eclipse.dev/Xtext/>

² <https://eclipse.dev/emf/>

observed during the animation of the B specification is reflected by Meeduse as an update of the AST; and since the grammar is defined in XText, these AST changes are automatically propagated back to the textual syntax of the model. This allows users to execute and debug DSL models at the language level, while all semantic reasoning is carried out in the formal framework of the B method.

As illustrated in Figure 2, transpilation is defined at the level of abstract syntax. A textual input file, conforming to the source grammar, is first parsed into a source AST. This AST is then represented in B by constants, reflecting the fact that the input model is fixed during transpilation. The target model, by contrast, is represented by variables, since it is progressively constructed by the transpiler. The transpilation itself is specified by B operations and invariants expressing the mapping between the source and target ASTs. By animating this B specification, the operations incrementally build the target AST from the source AST. Once this target AST has been obtained, it is serialized according to the target grammar, thereby yielding the output textual model.

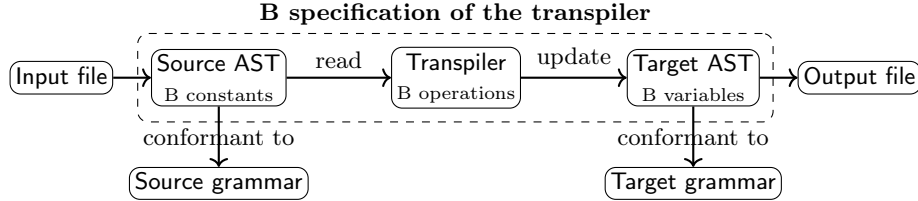


Fig. 2: Grammar-to-grammar transpilation in Meeduse.

3 The UML to B Use Case

For illustration we use the specification of a *Timer*, whose purpose is to trigger an alarm after a user-defined period of time. The system is controlled through a graphical interface (Figure 3) providing counters for hours, minutes and seconds, as well as control buttons such as *Mode*, *Incr*, and *On/Off*.



Fig. 3: Graphical Interface of the Timer System

3.1 System specification

The objective of this use case is to separate two concerns: (i) the control logic of the system, which describes how the system evolves between states in response to events, and (ii) the underlying data and operations, such as time manipulation,

which are encapsulated in a separate B machine. In this paper, we refer to the latter as the business logic, while the former corresponds to the controller. The business logic specification is shown in Figure 4 and can be obtained either from a UML-like class diagram [20] or specified directly in B. The controller is defined by the state machine of Figure 5. This diagram is defined in PlantUML, a textual modeling language that provides an UML graphical rendering.

```

MACHINE Timer VARIABLES seconds, minutes, hours, ring
INVARIANT
   $seconds \in 0 \dots 59 \wedge minutes \in 0 \dots 59 \wedge hours \in 0 \dots 23 \wedge ring \in 0 \dots 1$ 
INITIALISATION
   $seconds, minutes, hours, ring := 0, 0, 0, 0$ 
OPERATIONS
setRing = BEGIN  $ring := 1 - ring$  END;
setHr = BEGIN  $hours := (hours + 1) \bmod 24$  END;
setMn = BEGIN  $minutes := (minutes + 1) \bmod 60$  END;
update =
  BEGIN
     $seconds := (seconds + 59) \bmod 60 \parallel$ 
     $minutes := (minutes + 59 \times ((60 - seconds) \div 60)) \bmod 60 \parallel$ 
     $hours := (hours + 23 \times ((60 - minutes) \div 60) \times ((60 - seconds) \div 60)) \bmod 24$ 
  END;
END

```

Fig. 4: B Specification of the Timer Business Logic

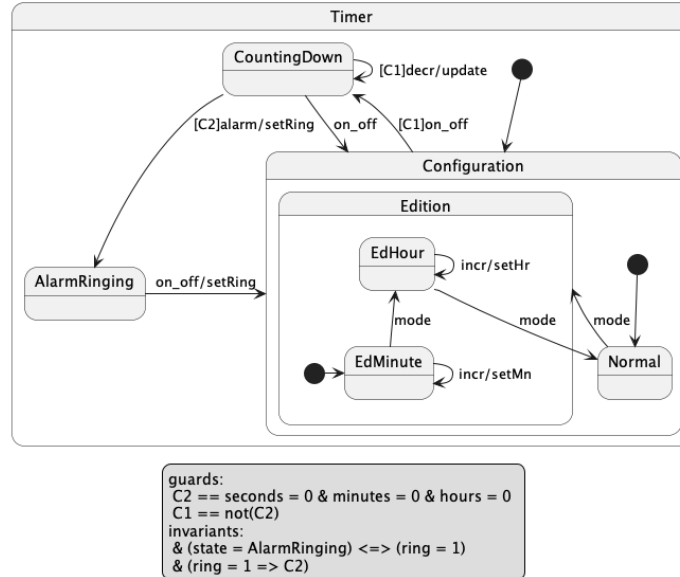


Fig. 5: State/Transition diagram of the controller defined with PlantUML

The Timer starts in state *Configuration*, which contains three sub-states: *Normal*, *EdMinute*, and *EdHour*. Event *mode* cycles through these sub-states in sequence. Event *incr* triggers the corresponding update operations: **setMn** in state *EdMinute* and **setHr** in state *EdHour*. Once the timer has been configured, the *on_off* event switches the system from *Configuration* to the *CountingDown* state, provided that condition [C1] holds, i.e., the counters are not all equal to zero. In the *CountingDown* state, the *decr* event periodically triggers the **update** operation, which decrements the counters according to the Timer business logic, as long as condition [C1] remains true. At any time during the countdown, the user may interrupt the process by pressing *On/Off*, which returns the system to the *Configuration* state. When the countdown reaches zero, condition [C2] becomes true and the system transitions to the *AlarmRinging* state, invoking operation **setRing**. Finally, the alarm can be stopped by the user via the *on_off* event, which calls again operation **setRing** and brings the system back to the *Configuration* state. The textual PlantUML specification of the controller is shown in Listing 1, which is the source model for our UML-to-B transpilation.

Listing 1: PlantUML textual specification of the Timer controller

```

1 @startuml
2 state Timer {
3   state Configuration {
4     state Normal
5     state Edition{
6       state EdMinute
7       state EdHour
8       [*] --> EdMinute
9     }
10    [*] --> Normal
11    Normal --> EdMinute : mode
12    EdMinute --> EdMinute : incr/setMn
13    EdMinute --> EdHour : mode
14    EdHour --> EdHour : incr/setHr
15    EdHour --> Normal : mode
16  }
17  state CountingDown
18  state AlarmRinging
19  [*] --> Configuration
20  Configuration --> CountingDown : [C1] on_off
21  CountingDown --> Configuration : on_off
22  CountingDown --> CountingDown : [C1] decr/update
23  CountingDown --> AlarmRinging : [C2] alarm/setRing
24  AlarmRinging --> Configuration : on_off/setRing
25 }
26 legend
27   guards: == ...
28   invariants: & ...
29 end legend
30 @enduml

```

3.2 Formal B model of the controller

Several works have addressed the translation of UML state machines into B. In this work, we adopt a similar approach being inspired by [20]. The structural part of the resulting B specification is presented in Figure 6.

```

MACHINE Controller INCLUDES Timer
SETS
  State = {Normal, EdMinute, EdHour, CountingDown, AlarmRinging}
DEFINITIONS
  C2 == (seconds = 0 ∧ minutes = 0 ∧ hours = 0) ;
  C1 == (not(C2)) ;
  Configuration == {Normal} ∪ Edition ;
  Edition == {EdMinute, EdHour}
VARIABLES state
INVARIANT
  state ∈ State
  ∧ ((state = AlarmRinging) ⇔ (ring = 1))
  ∧ (ring = 1 ⇒ C2)
INITIALISATION
  state := Normal

```

Fig. 6: Header of the B Specification of the Timer Controller

This machine includes the previously defined **Timer** machine. It introduces the set **State**, which enumerates the control states of the controller, and a variable **state** that records the current control state. The remaining parts of the invariant are directly extracted from the **legend** section of the PlantUML model: the alarm is ringing (*ring* = 1) if and only if the controller is in state **AlarmRinging**, and this state can only be reached when condition **C2** holds. The initialization assigns value **Normal** to variable **state**.

Figure 7 presents the operational part. Each operation corresponds to a UML event and is encoded as a **SELECT** statement whose branches represent the transitions triggered by that event. Each branch specifies a source state, an optional guard condition, and the associated effects. Executing an operation consists in selecting one enabled branch, which updates the current state and/or invokes the corresponding operations of the underlying model, thereby reflecting the transition semantics of the source state machine.

4 Transpilation

This section presents the grammar-to-grammar transpilation framework implemented in **Meeduse**. It introduces the source and target languages, their B formal abstract syntax, and the principles underlying their semantic mapping.

4.1 Source language

The grammar of the input language is presented in Listing 2. It is tailored to serve as the source language for our transpilation process. In this grammar the root

```

mode = SELECT state = Normal THEN state := EdMinute
      WHEN state = EdMinute THEN state := EdHour
      WHEN state = EdHour THEN state := Normal
      END;

incr = SELECT state = EdMinute THEN setMn
      WHEN state = EdHour THEN setHr
      END;

on_off =
  SELECT state ∈ Configuration ∧ C1 THEN state := CountingDown
  WHEN state = CountingDown THEN state := Normal
  WHEN state = AlarmRinging THEN setRing || state := Normal
  END;

decr = SELECT state = CountingDown ∧ C1 THEN update END;

alarm = SELECT state = CountingDown ∧ C2 THEN
        setRing || state := AlarmRinging
        END

```

Fig. 7: B operations of the Controller

element is a composite state, which represents the top-level state machine and may contain nested states and transitions. States are either simple (*SSimple*) or composite (*SComposite*). A composite state encloses a set of sub-states together with the transitions that originate from or target these states. This structure allows hierarchical state machines, in line with standard UML semantics. Simple states, by contrast, have no internal structure. Regarding transitions, they are classified into two categories: (i) start transitions (*TStart*) originating from the initial pseudo-state [***], and (ii) event-triggered transitions (*TEvent*) connecting two states. Such transitions may optionally be guarded and may invoke an action, following the UML notation [guard]event/action.

In addition to the structural elements of state machines, the language supports the specification of semantic information through an optional **legend** section. This section allows guards and invariants to be declared textually as uninterpreted expressions (*i.e.*, free text). Overall, this source language focuses on a restricted yet expressive fragment of UML state/transition diagrams. The textual specification shown in Listing 1 is a valid model conforming to this grammar.

Listing 2: XText grammar of the input language

```

1 STModel : '@startuml'
2           root=SComposite
3           ('legend'
4             'guards:' (guard+=TGuard)*
5             'invariants:' (invs+=INV)*
6             'end' 'legend')?
7           '@enduml' ;
8 State : SComposite | SSimple ;
9 SComposite : 'state' name=ID '{'

```



```

10         (states+=State)+
11         (transitions+=Transition)+
12     '}' ;
13 SSimple : 'state' name=ID ;
14 Transition : TStart | TEvent ;
15 TStart : '[' '*' '-->' target=[State] ;
16 TEvent : source=[State] '-->' target=[State] ':'
17         ('[' guard=[TGuard] ']' )? name=ID ('/' action=ID)? ;
18 TGuard : name=ID val=GUARD ;
19 terminal INV : '&' !('\n' | '\r')* ;
20 terminal GUARD : '==' !('\n' | '\r')* ;

```

4.2 Target language

The grammar of the target language is presented in Listing 3. Rather than relying on the full syntax of the B method, we introduce a domain-specific subset of B tailored to represent the target models produced by the transpilation. This subset captures the structure and behavior of B controllers derived from UML state machines. The root element of the grammar is a **BMachine**, which may include a set of states, guard definitions, state definitions, variables, invariants, and operations. Simple states are defined as elements of set **State**, while state definitions are for composite states. Guard definitions provide reusable conditions that can be referenced in events, which are composed of branches, each having a source state, an optional guard condition, and actions. The latter either update the current state or invoke B operations. The B machine shown in Figures 6 and 7 is a valid model conforming to this grammar.

Listing 3: XText grammar of the B controller language

```

1 BMachine :
2   'MACHINE' name=ID ('INCLUDES' inc=ID)?
3   ('SETS' 'State' '=' '{' elems+=StateId (',' elems+=StateId)* '}' )?
4   ('DEFINITIONS'
5     (grdDefs+=GuardDef)* (stDefs+=StateDef (',' stDefs+=StateDef)* )?
6   )?
7   ('VARIABLES' 'state'
8     'INVARIANT' 'state' ':' 'State' (invs+=INV)*
9     'INITIALISATION' 'state' ':' initS=[StateId])?
10  ('OPERATIONS' events+=BEvent (',' events+=BEvent)* )?
11  'END' ;
12 StateId : name=ID ;
13 StateDef : name=ID '==' '{' refs+=[StateId] (',' refs+=[StateId])* '}'
14           ('\n' unionWith+=[StateDef])* ;
15 GuardDef : name=ID '==' val=GUARD ;
16 BEvent : name=ID '=' 'SELECT' branches+=Branch
17         ('WHEN' branches+=Branch)* 'END' ;
18 Branch : from=BState ('&' cond=[GuardDef])?
19         'THEN' subs+=BAction ('||' subs+=BAction)* ;
20 BState : 'state' ('=' simple=[StateId] | ':' composite=[StateDef]) ;

```

```

21 BAction : 'state' ':' to=[StateId] | op=ID ;
22 terminal GUARD : '(' -> ')';
23 terminal INV : '&' ( ' ' | '\t' )* '(' !('\n' | '\r')* ;

```

4.3 Formal abstract syntax

Given an Xtext grammar, Meeduse automatically extracts a formal B specification defining the abstract syntax of the DSL according to a generic scheme. A non-terminal Nt is translated into an abstract set NT , which characterizes the set of all admissible instances of the corresponding construct. The instances that actually occur in a model are represented by a constant (or a variable) Nt , together with the property (or invariant) $Nt \in \mathcal{F}(NT)$. Figure 8 presents the **PROPERTIES** clause generated by Meeduse and defining the abstract syntax of the source language. Elements in capital letters are declared in the **SETS** clause as abstract sets and the other ones are introduced in the **CONSTANTS** clause.

```

PROPERTIES
/* STModel */
  STModel  $\in \mathcal{F}(STMODEL) \wedge invs \in STModel \leftrightarrow STR$ 
/* State */
   $\wedge State \in \mathcal{F}(STATE)$ 
   $\wedge SComposite \subseteq State \wedge SSimple \subseteq State \wedge SSimple \cap SComposite = \emptyset$ 
   $\wedge State\_name \in State \rightarrow STR \wedge states \in State \mapsto SComposite$ 
   $\wedge root \in STModel \mapsto SComposite$ 
/* Transition */
   $\wedge Transition \in \mathcal{F}(TRANSITION)$ 
   $\wedge TEvent \subseteq Transition \wedge TStart \subseteq Transition \wedge TEvent \cap TStart = \emptyset$ 
   $\wedge TEvent\_name \in TEvent \rightarrow STR$ 
   $\wedge transitions \in Transition \mapsto SComposite$ 
   $\wedge tsource \in TEvent \rightarrow State \wedge ttarget \in TEvent \rightarrow State$ 
   $\wedge taction \in TEvent \mapsto STR \wedge starget \in TStart \rightarrow State$ 
/* TGuard */
   $\wedge TGuard \in \mathcal{F}(TGUARD)$ 
   $\wedge val \in TGuard \rightarrow STR \wedge TGuard\_name \in TGuard \rightarrow STR$ 
   $\wedge guard \in TGuard \rightarrow STModel \wedge tguard \in TEvent \mapsto TGuard$ 

/* contextual constraints of the input language */
dom(states) = State - ran(root)  $\wedge$ 
(TStart  $\triangleleft$  transitions)  $\in TStart \mapsto SComposite \wedge$ 
card(ran(State\_name)) = card(State)

```

Fig. 8: Formal abstract syntax of the source language

Mutually exclusive grammar alternatives lead to disjoint subsets, as for rules **State** and **Transition**. Grammar attributes are translated into B relations whose specialization depends on the nature of the attribute: mandatory single-valued attributes induce total functions (e.g., **State_name**), optional attributes

induce partial functions, and multi-valued references lead to relations or surjective mappings (*e.g.*, **states** or **transitions**).

Contextual constraints that cannot be directly expressed at the grammar level are captured through hand-written properties or invariants. Figure 8 shows three such contextual constraints of the input language, which ensure that: (i) every state (except the root) is owned by exactly one composite state; (ii) every composite state has exactly one initial state; and (iii) state names are unique. We recall that any syntactically valid instance of the grammar is translated by *Meeduse* into a refinement of the abstract syntax specification. If such a refinement is produced from a model that is syntactically correct but violates one or many contextual constraints, *ProB* detects the inconsistency by failing the corresponding predicates. *Meeduse* then propagates this feedback and reports which property is violated and which model elements are involved.

Figure 9 presents the abstract syntax of the target B language, following the same conventions as for the source language.

INVARIANT <i>/* Core generated sets */</i> $BMachine \in \mathcal{F}(BMACHINE)$ $\wedge StateId \in \mathcal{F}(STATEID)$ $\wedge GuardDef \in \mathcal{F}(GUARDDEF)$ $\wedge StateDef \in \mathcal{F}(STATEDEF)$ $\wedge BEvent \in \mathcal{F}(BEVENT)$ $\wedge Branch \in \mathcal{F}(BRANCH)$ $\wedge BState \in \mathcal{F}(BSTATE)$ $\wedge BAction \in \mathcal{F}(BACTION)$ <i>/* Ownership relations */</i> $\wedge elems \in StateId \rightarrow BMachine$ $\wedge grdDefs \in GuardDef \rightarrow BMachine$ $\wedge stDefs \in StateDef \rightarrow BMachine$	$\wedge events \in BEvent \rightarrow BMachine$ <i>/* State structure */</i> $\wedge simple \in BState \leftrightarrow StateId$ $\wedge composite \in BState \leftrightarrow StateDef$ $\wedge refs \in StateDef \leftrightarrow StateId$ $\wedge unionWith \in StateDef \leftrightarrow StateDef$ $\wedge initS \in BMachine \leftrightarrow StateId$ <i>/* Control structure */</i> $\wedge from \in Branch \rightarrow BState$ $\wedge subs \in BAction \rightarrow Branch$ $\wedge branches \in Branch \rightarrow BEvent$ $\wedge cond \in Branch \leftrightarrow GuardDef$ $\wedge to \in BAction \leftrightarrow StateId$ $\wedge op \in BAction \leftrightarrow STR$
<i>/* Naming */</i> $\wedge StateId_name \in StateId \rightarrow STRING$ $\wedge StateDef_name \in StateDef \rightarrow STRING$ $\wedge GuardDef_name \in GuardDef \rightarrow STRING$ $\wedge BEvent_name \in BEvent \rightarrow STRING$ $\wedge bval \in GuardDef \rightarrow STRING$ $\wedge binvs \in BMachine \leftrightarrow STRING$ $\wedge inc \in BMachine \leftrightarrow STRING$	

Fig. 9: Abstract syntax and structural invariants of the generated B machine

The transpilation relies on a restricted B grammar whose constructs are sufficient to represent the target B machines produced by the translation. This restriction is not merely syntactic; it also allows structuring the transpilation and making its correctness easier. In particular, dedicated semantic elements such as

BState and **BAction** provide an explicit correspondence between concepts of the source language and their counterparts in the target formalism. As a result, the semantics of the target B specification is not introduced independently, but is systematically derived from the semantics of the source formalism.

4.4 Transpilation properties

Unlike compilers, transpilation is a source-to-source transformation where both languages are at the same abstraction level. As the B method is used to specify the transpiler, we propose to first identify and formalise the key properties that must be preserved. By expressing these properties as invariants, the transpilation is guided by correctness criteria defined *a priori*, rather than being validated *a posteriori*. Below, we present a non exhaustive list of such invariants, having a variable bm such that $bm \in BMachine$, and a constant st such that $st \in STModel$, representing respectively the target B machine and the source UML diagram:

(inv_1)	$(binvs \neq \emptyset \Rightarrow \mathbf{ran}(binvs) = invs[\{st\}])$
(inv_2)	$elems = StateId \times \{bm\}$ $\wedge (elems \neq \emptyset \Rightarrow \mathbf{card}(StateId) = \mathbf{card}(SSimple)$ $\quad \wedge StateId_name[StateId] = \mathbf{ran}(SSimple \triangleleft State_name))$
(inv_3)	$(grdDefs = GuardDef \times \{bm\})$ $\wedge (grdDefs \neq \emptyset \Rightarrow \mathbf{card}(GuardDef) = \mathbf{card}(guard^{-1}[\{st\}])$ $\quad \wedge GuardDef_name[GuardDef] = \mathbf{ran}(guard^{-1}[\{st\}] \triangleleft TGuard_name)$ $\quad \wedge bval[GuardDef] = \mathbf{ran}(guard^{-1}[\{st\}] \triangleleft val))$
(inv_4)	$(stDefs = StateDef \times \{bm\})$ $\wedge \mathbf{ran}(StateDef_name) \subseteq \mathbf{ran}(SComposite \triangleleft State_name)$ $\wedge \mathbf{ran}(refs) \subseteq StateId_name^{-1}[\mathbf{ran}(SSimple \triangleleft State_name)]$ $\wedge StateDef_name[\mathbf{dom}(unionWith) \cup \mathbf{ran}(unionWith)]$ $\quad \subseteq \mathbf{ran}(SComposite \triangleleft State_name)$

Invariant (inv_1) ensures that the invariants of the generated B machine coincide with those of the source model whenever they are present. Invariant (inv_2) establishes a one-to-one correspondence between simple states in the source model and their representations in the generated B machine. Invariant (inv_3) plays an analogous role for guards, ensuring that guard names and expressions are preserved without loss or duplication during transpilation. Finally, invariant (inv_4) ensures that each generated state definition is named after a composite state of the source model. References inside state definitions are restricted to identifiers corresponding to simple sub-states of the source composite state, and union relationships between state definitions only relate composite states, thereby preserving the hierarchical structure of states.

The transpiler must also enforce well-formedness constraints on the generated B specification. The following invariants illustrate some examples of how the translation constrains the generated actions, states, and branches.

(<i>inv</i> ₅)	$\mathbf{dom}(to) \cap \mathbf{dom}(op) = \emptyset$ $\wedge \mathbf{dom}(to) \cup \mathbf{dom}(op) = BAction$
(<i>inv</i> ₆)	$\mathbf{dom}(simple) \cap \mathbf{dom}(composite) = \emptyset$ $\wedge \mathbf{dom}(simple) \cup \mathbf{dom}(composite) = BState$
(<i>inv</i> ₇)	$Branch \subseteq \mathbf{dom}(branches) \cap \mathbf{ran}(subs) \cap \mathbf{dom}(from)$
(<i>inv</i> ₈)	$\mathbf{dom}(to) \subseteq \mathbf{dom}(subs)$ $\wedge \mathbf{dom}(op) \subseteq \mathbf{dom}(subs)$
(<i>inv</i> ₉)	$\mathbf{ran}(from) \subseteq \mathbf{dom}(simple) \cup \mathbf{dom}(composite)$

Invariant (*inv*₅) enforces a disjoint alternative between the two forms of B actions: an action is either a state update (*to*) or an operation call (*op*), but never both, and guarantees that every generated action is classified. Invariant (*inv*₆) plays an analogous role for B states: each generated state is either simple or composite, but never both, and the union of these two categories covers the entire set of B states. Invariant (*inv*₇) captures the structural properties of branches. It ensures that every branch is properly attached to an event (*branches*), has a defined source state (*from*), and is associated with at least one action via the substitution relation (*subs*). Invariant (*inv*₈) strengthens this property by requiring that all generated actions, whether state updates or operation calls, are attached to an existing branch. Finally, invariant (*inv*₉) guarantees that the source state of every branch is a valid B state, either simple or composite.

4.5 Grammar-to-grammar transpiler

Having defined the abstract syntax of both source and target languages together with their properties, we now focus on the semantic mapping that underpins the transpilation process. To this purpose, we have investigated two versions of the transpiler. In the first one, all transformation steps were defined in a single machine. This monolithic design enabled a prototype validation of the core transpilation principles and the main structural and semantic invariants. Although this first version was fully proved, it quickly revealed limitations in terms of readability, modularity, and proof scalability, with a total of 321 proof obligations (POs), among which 26 required interactive proofs. In particular, the tight interleaving of the output language updates with high-level semantic decisions made the specification and the POs harder to understand and maintain.

To address these issues, we designed a second version of the transpiler with a clearer architectural separation. In this version, the transpilation is structured around two machines: a **utility** machine that encapsulates all primitive language mutations (*i.e.*, simple/composite state creation, event generation, branch linking, update of guards and actions), and a dedicated **transpiler** machine that expresses only the semantic logic of the UML-to-B translation. This separation clearly distinguishes *what* is transformed (basic operations on the output language) from *how* it is done (semantic decisions), resulting in simpler and more manageable POs. Figure 10 reports the corresponding Atelier B proof statistics.

Component Status for transpiler					
AutoProved /Users/idani/Documents/AB/U2BCert/transpiler.mch					
	nPO	nPRI	nPRa	nUn	%Pr
AssertionLemmas	11	0	11	0	100
Initialisation	21	0	21	0	100
WellDefinednessProperties	1	0	1	0	100
WellDefinedness_addEventBranchExisting	28	0	28	0	100
WellDefinedness_addEventBranchFresh	26	0	26	0	100
WellDefinedness_linkUnionWith	6	0	6	0	100
addBranchAction	9	2	7	0	100
addBranchGuard	2	1	1	0	100
addEventBranchExisting	20	5	15	0	100
addEventBranchFresh	24	3	21	0	100
compositeState	9	2	7	0	100
guardDefs	5	0	5	0	100
inclusion	1	0	1	0	100
initState	1	0	1	0	100
invariants	2	0	2	0	100
linkUnionWith	1	0	1	0	100
simpleStates	5	0	5	0	100
transpiler	172	13	159	0	100

Fig. 10: Atelier B proof statistics of the U2Bcert transpiler

The **utility** machine consists of approximately 400 lines of B code, from which Atelier B generated 89 POs, all of which were discharged automatically. Roughly half of this machine corresponds to declarations of constants and variables and typing invariants and properties automatically generated by **Meeduse**. The **transpiler** machine is also fully proved as shown in Figure 10. In the remainder, we present excerpts of the main ideas defined across these two machines. For more details, we made all artefacts of this work (grammars, use-case models, B specifications, etc) publicly available at <https://github.com/meeduse/u2bcert>.

Creation of simple states in the B machine. The B operation **simpleStates**, shown in Figure 11, defined in the **transpiler** machine, captures the semantic decision of allocating one target state identifier per UML simple state. It is enabled only when no target artefact has yet been generated, ensuring that this phase is executed exactly once and before any transition, guard, or action is introduced. The operation selects a finite set of fresh identifiers whose cardinality matches the number of simple states in the source model, and establishes a total bijection between these identifiers and the names of UML simple states. The construction of the target artefacts is delegated to the utility operation **setSimpleStates**, which performs the corresponding updates on the target B machine. When applied to our running example, this step instantiates the grammar rule:

```
('SETS' 'State' '=' '{' elems+=StateId (',' elems+=StateId)* '}'')
```

resulting in the generation of the clause **SETS** shown below:

```
SETS
  State = { AlarmRinging, CountingDown, EdHour, EdMinute, Normal }
```

```

/* from utility machine */
setSimpleStates(bMachine, stIds, nameMap) =
  PRE
    bMachine ∈ BMachine ∧ stIds ∈  $\mathcal{F}(\text{STATEID})$ 
    ∧ nameMap ∈ stIds → STRING
    ∧ elems = ∅ ∧ refs = ∅ ∧ simple = ∅ ∧ to = ∅ ∧ initS = ∅
  THEN
    StateId := stIds ||
    elems := stIds × {bMachine} ||
    StateId_name := nameMap
  END;

/* from transpiler machine */
simpleStates =
  SELECT
    elems = ∅ ∧ refs = ∅ ∧ simple = ∅ ∧ to = ∅ ∧ initS = ∅
  THEN
    ANY stIds, nameMap WHERE
      stIds ∈  $\mathcal{F}(\text{STATEID})$  ∧ card(stIds) = card(SSimple)
      ∧ nameMap ∈ stIds ↦ ran(SSimple < State_name)
    THEN
      setSimpleStates(bm, stIds, nameMap)
    END
  END;

```

Fig. 11: Creation of simple states in the target B machine

Identification of the initial state. The initialization of the target B machine is performed after the creation of simple states and before any transition is generated. This step determines the initial control state of the B machine from the initial configuration of the UML state-transition diagram. The operation **initState** of Figure 12, expresses the underlying semantic choice. It selects a simple UML state that is reachable from the root composite state through an initial transition, and maps it to its corresponding *StateId* in the target machine. The actual update of the target B machine is delegated to the utility operation **setInit**.

The definition of relation *initRel* is given below. It captures the semantics of UML initial transitions by associating each composite state with the simple states that can be reached from it via a *TStart* transition.

$$\begin{aligned}
 \text{initRel} == \{ & cc, ss \mid cc \in S\text{Composite} \\
 & \wedge (\exists ts \cdot (ts \in T\text{Start} \wedge ts \in \text{transitions}^{-1} [\{cc\}] \\
 & \wedge ss = \text{starget}(ts))) \};
 \end{aligned}$$

Operation **initState** selects one such simple state and retrieves its corresponding *StateId* in the target B machine. This *StateId* is then set as the initial state of the B machine through the utility operation **setInit**. When applied to our running example, it results in the following fragment of the target B machine:

```

VARIABLES
  state
INVARIANT
  state : State
INITIALISATION
  state := Normal

```

```

/* from utility machine */
setInit(bMachine, sid) =
  PRE
    bMachine ∈ BMachine ∧ sid ∈ StateId
  THEN
    initS(bMachine) := sid
  END;

/* from transpiler machine */
initState =
  SELECT
    elems ≠ ∅ ∧ initS = ∅
  THEN
    ANY sm, sid WHERE
      sm ∈ SSimple ∧ sid ∈ StateId
      ∧ sm ∈ (closure1(initRel)) [{root(st)}] - dom(initRel)
      ∧ StateId_name(sid) = State_name(sm)
    THEN
      setInit(bm, sid)
    END
  END;

```

Fig. 12: Identification of the initial state in the target B machine

Note that the remaining transpilation steps follow the same overall pattern. The **transpiler** machine consists of 372 lines of B code. It defines 11 B operations, together with the invariants presented in the previous sub-section. This structure provides a fully proved, executable, and modular grammar-to-grammar mappings that translate UML state-transition diagrams into B machines.

5 Related works

Our proposal provides benefits to the numerous studies conducted within the B community on the formalization of semi-formal notations. For instance, [18] formalizes the operational semantics of UML2.x sequence diagrams, while [17] proposes a run-to-completion semantics for state/transition diagrams, among others. Not only could the operational semantics defined in such works be instrumented and executed within Meeduse, but also the underlying model-to-B transformation, that we revisit by means of grammar-to-grammar transpilation.

However, existing tools that generate B from semi-formal notations systematically leave the correctness proof of their transformations as future work; and to the best of our knowledge, such proofs have never been achieved. This is a major limitation, since the generated B specifications are intended to serve as verification artifacts for the source models: if the transformation itself is incorrect, one may end up performing complex verification tasks on invalid specifications. Moreover, these tools usually implement their transformations in third-party technologies such as Java or ATL, which makes it harder to establish their correctness. By contrast, *Meeduse* allows transformation rules to be directly formalized in B, their correctness proved using B’s proof tools, and then executed within the same environment.

Beyond the B community, other formal methods have been applied to the verification of DSLs and transpilers. A prominent example is Isabelle/HOL [19], which has even been used to formalize and prove the correctness of compilers. Approaches based on Isabelle/HOL benefit from the expressive power of higher-order logic and the maturity of the Isabelle proof environment. However, the resulting specifications are not directly executable on models: additional mechanisms such as code generation or integration with external languages are required to obtain an operational artifact [5,16]. This contrasts with our approach in *Meeduse*, where the very same B specifications are both verified and executed.

6 Conclusion

This chapter has presented an approach for formally verified grammar-to-grammar transpilation within the *Meeduse* LWB. The approach has been illustrated through a use case involving the translation of UML state/transition diagrams into formal B specifications. The transpilation process is defined using B operations that map the abstract syntax of the source language to that of the target language. By animating the B specification based on a given AST, the transpilation can be executed, thereby producing the AST of the output model.

Our formal framework is not intended to reveal inconsistencies during transpilation, but rather to prevent them by construction through theorem proving. Instead of detecting errors a posteriori, the approach guarantees that any generated target model satisfies the specified invariants, provided that the source model itself is well-formed. Nevertheless, the formalisation plays a crucial role in ruling out several classes of potential errors. For instance, inconsistencies in the source model (*e.g.*, ill-formed state hierarchies or missing initial states) are detected through violations of contextual constraints. Errors in the transpilation logic, such as missing mappings, incorrect state correspondences, or incomplete handling of transitions, are excluded by construction: such issues would lead to POs that cannot be discharged, thereby preventing the validation of the specification. Since all POs have been successfully discharged, the transpilation is guaranteed to preserve the specified invariants for any well-formed source model. Similarly, inconsistencies in the construction of the target abstract syntax (*e.g.*, dangling references, malformed branches, or ill-typed actions) cannot arise in

the validated specification. These properties are enforced by the structural invariants of the target language, whose preservation is formally established by the proof. In this sense, the contribution of the approach is not to detect errors during a posterior validation phase, but to guarantee their absence through a fully proved and executable specification.

Several research directions emerge from this work. First, the proposed framework opens the way to formally verified chains of transpilation, where multiple grammar-to-grammar mappings are composed within a single B specification, allowing end-to-end reasoning across successive translations. Second, the current approach could be extended to support semantic refinement between source and target languages, making it possible to capture translations that introduce additional structure or behavioral detail while preserving invariant properties. Finally, integrating other state-based formalisms, such as ASM or TLA⁺, as either source or target languages, would allow investigating interoperability between heterogeneous formal methods within a single framework.

Acknowledgments. The author would like to thank German Vega for his great technical expertise and contribution to Meeduse, and Sylvain Boulmé for his valuable feedback on earlier drafts of this chapter.

References

1. J.-R. Abrial. *The B-book: Assigning Programs to Meanings*. Cambridge University Press, 1996.
2. J.-R. Abrial. *Modeling in Event-B: System and Software Engineering*. Cambridge University Press, 2010.
3. J.-R. Abrial. On B and Event-B: Principles, success and challenges. In *6th International Conference on Abstract State Machines, Alloy, B, TLA, VDM, and Z (ABZ)*, volume 10817 of *LNCS*, pages 31–35. Springer, 2018.
4. J.-R. Abrial, M. Butler, S. Hallerstede, T. S. Hoang, F. Mehta, and L. Voisin. Rodin: an open toolset for modelling and reasoning in Event-B. *International Journal on Software Tools for Technology Transfer*, 12(6):447–466, 2010.
5. M. Haslbeck and P. Lammich. Unifying model execution and deductive verification with interaction trees in isabelle/hol. *arXiv preprint*, 2024.
6. A. Idani. Meeduse: A tool to build and run proved DSLs. In *16th Intl Conf. on Integrated Formal Methods*, volume 12546 of *LNCS*, pages 349–367. Springer, 2020.
7. A. Idani. Formal model-driven executable DSLs: Application to Petri-Nets. *Innovations in Systems and Software Engineering (ISSE)*, 18(1), 2022.
8. A. Idani. The B Method meets MDE: Survey, progress and future. In *16th Intl. Conference on Research Challenges in Information Science (RCIS)*, volume 446, pages 495–512. Springer, 2022.
9. A. Idani, Y. Ledru, and G. Vega. Alliance of Model Driven Engineering with a Proof-based Formal Approach. *Innovations in Systems and Software Engineering*, 16(3):289–307, 2020.
10. A. Idani, Y. Ledru, A. A. Wakrime, R. B. Ayed, and P. Bon. Towards a tool-based domain specific approach for railway systems modeling and validation. In *International Conference on Reliability, Safety, and Security of Railway Systems (RSSRail)*, volume 11495 of *LNCS*, pages 23–40. Springer, 2019.

11. A. Idani, G. Vega, and M. Leuschel. Applying formal reasoning to model transformation: The meeduse solution. In *Proceedings of the 12th Transformation Tool Contest, co-located with STAF'2019, Software Technologies: Applications and Foundations*, volume 2550, pages 33–44, 2019.
12. A. Iung, J. Carbonell, L. Marchezan, E. M. Rodrigues, M. Bernardino, F. P. Basso, and B. Medeiros. Systematic mapping study on domain-specific language development tools. *Empirical Software Engineering*, 25(5):4205–4249, 2020.
13. T. Kosar, S. Bohra, and M. Mernik. Domain-specific languages: A systematic mapping study. *Information and Software Technology*, 71:77–91, 2016.
14. T. Lecomte, D. Deharbe, E. Prun, and E. Mottin. Applying a formal method in industry: A 25-year trajectory. In *Formal Methods: Foundations and Applications*, volume 10623 of *LNCS*, pages 70–87. Springer, 2017.
15. M. Leuschel and M. Butler. ProB: an automated analysis toolset for the B method. *Software Tools for Technology Transfer (STTT)*, 10(2):185–203, 2008.
16. M. Meghzili, J. Combaz, and J. Sifakis. Verification of model transformations using isabelle/hol and scala. In *International Conference on Software Engineering and Formal Methods (SEFM)*, pages 200–215, 2018.
17. K. Morris, C. F. Snook, T. S. Hoang, G. C. Hulet, R. Armstrong, and M. J. Butler. Formal Verification of Run-to-Completion Style Statecharts Using Event-B. In *14th European Conference on Software Architecture*, volume 1269 of *CCIS*, pages 311–325. Springer, 2020.
18. I. Mouakher, F. Dhaou, and J.-C. Attiogbé. Event-Based Semantics of UML 2.X Concurrent Sequence Diagrams for Formal Verification. *Journal of Computer Science and Technology*, 37(1):4–28, 2022.
19. T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL — A Proof Assistant for Higher-Order Logic*, volume 2283 of *LNCS*. Springer, 2002.
20. C. Snook and M. Butler. UML-B: Formal modeling and design aided by UML. *ACM Transactions on Software Engineering and Methodology*, 15(1), 2006.